

適応型認証における Guard 付き Criterion Trajectory : 現在の認証判断と将来の認証基準更新の分離

川上俊太郎

目次

要旨	2
1 はじめに	2
2 背景と Related Work	4
2.1 GyroAuth の基礎	4
2.2 Digital authentication と Zero Trust	4
2.3 Continuous authentication と Behavioral biometrics	4
2.4 Concept drift と Poisoning	5
3 用語と適用範囲	5
3.1 Access Subject	5
3.2 Expected Identity	6
3.3 Observed Access Trajectory	6
3.4 Authentication Criterion	6
3.5 Criterion Trajectory と Criterion Integrity	6
4 Threat Model	6
5 Formal Security Model	7
6 Criterion Update State Machine	9
7 PoC 設計	10
7.1 Model U: Direct adoption	10
7.2 Model G: Guarded adoption	10
8 結果	11
8.1 N1: 正当な新規端末移行	11
8.2 P1: 段階的な許容領域拡張 Poisoning	11
8.3 C1: 単一 Evidence source 侵害	13
9 考察	14
10 Security Claims と限界	15

11	再現性と公開物	15
12	結論	15
13	参考文献	16
14	AI 支援ツールの使用	17
15	申告事項	17

所属: 個人

ORCID: 0009-0004-0091-1303

責任著者連絡先: dev.jxiv@gyro-wedge.com

キーワード: 適応型認証; 継続認証; Criterion poisoning; Guard 付き適応; Criterion Integrity; Trajectory; GyroAuth

要旨

適応型認証は、端末、ネットワーク、場所、役割、行動などの正当な変化へ対応しなければならない。認証基準が一切変化しなければ硬直的になる一方、観測された挙動を独立した制御なしに取り込む認証基準は、追加の攻撃面を生む。すなわち、不正な挙動が段階的に新しい通常状態として吸収される可能性がある。本稿は、認証基準の変化を自動的なプロファイル更新としてではなく、Guard 付き Criterion Trajectory として扱う GyroAuth 拡張を提案する。本モデルは、現在の認証判断と、将来の認証に用いる基準を変更する判断とを分離する。Criterion Update Candidate は、現在の Criterion、Observed Access Trajectory、Context、Evidence、History から生成されるが、Guard を経て、Criterion Update Response として ACCEPT、DEFER、FREEZE、REVIEW、ROLLBACK のいずれかが選択された後にのみ有効化される。本稿では、Subject Evaluation と Criterion Integrity Evaluation を分離し、有限な Criterion Update State Machine を定義し、Candidate の直接採用モデルと Guard 付き採用モデルを比較する決定論的 PoC を実装した。正当な新規端末シナリオでは、Guard 付きモデルは Challenge 確認まで更新を保留し、その後、限定的な更新を採用した。段階的な許容領域拡張 Poisoning では、直接更新モデルが攻撃参照値を許容するまで拡張した一方、Guard 付きモデルは許容前に更新を凍結した。単一 Evidence source 侵害では、自動採用を阻止した。これらの結果は Synthetic な仮定下で提案構造が実行可能であることを示すが、本番環境の安全性、普遍的な攻撃検知、False Accept 率や False Reject 率を保証するものではない。

キーワード: 適応型認証、継続認証、Criterion poisoning、Guard 付き適応、Criterion Integrity、Trajectory、GyroAuth

1 はじめに

認証は、端末、ネットワーク、場所、行動、運用 Context が時間とともに変化する環境で実行される。Digital authentication guidance は Authenticator や Assurance の要件を定めるが、時間を通じて取得される観測を将来の認証判断へどのように反映するかは、Application layer で別途設計しなければならない [1], [2]。

固定された Criterion は正当な変化を拒否し得る一方、無制約な適応は攻撃者による挙動の正常化を許し得る。

本研究の中心命題は次である。

dynamic criterion

!=

unconstrained self-update

したがって、問題は二重である。GyroAuth は、現在の Access Subject が Expected Identity との関係で許容可能かを評価すると同時に、その判断に用いる Authentication Criterion 自体が引き続き許容可能かを、別の判断として評価しなければならない。

Subject Evaluation

!=

Criterion Integrity Evaluation

判断空間も分離する。

Auth Decision

!=

Criterion Update Response

さらに、更新候補と正式な基準も分離する。

Criterion Update Candidate

!=

Accepted Criterion

この分離により、次の組合せを実行可能にする。

AUTH_STABLE + FREEZE

現在の Authentication Relation は一時的に継続可能である一方、将来の Criterion 適応は停止できる。

本稿の貢献は次の通りである。

1. Subject Evaluation と Criterion Integrity Evaluation を分離する二重評価モデル。
2. Criterion 変化を追跡可能にする Criterion Trajectory 表現。
3. ACCEPT、DEFER、FREEZE、REVIEW、ROLLBACK からなる Guard 付き Criterion Update。
4. 有限な Criterion Update State Machine。
5. Candidate 直接採用と Guard 付き採用を比較する決定論的 PoC。
6. Security assumption、支持される主張、限界、非保証の明示。

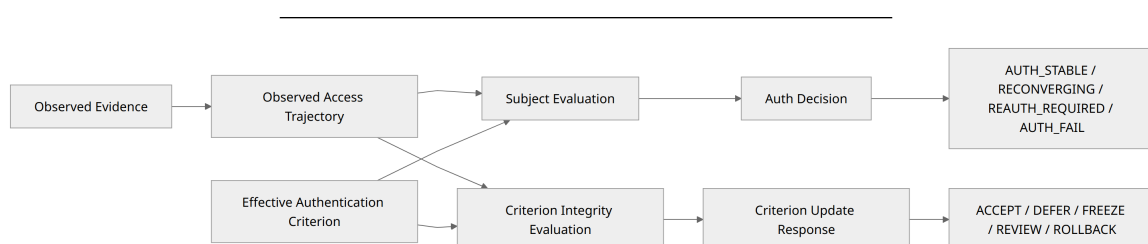


図 1. GyroAuth は、現在の Authentication Relation と、将来の評価に用いる Criterion の Integrity を、関連しつつも別の Process として評価する。

2 背景と Related Work

2.1 GyroAuth の基礎

GyroAuth は認証を次のように定義する。

Authentication

=

Stability-based Selection over State Convergence

レイヤー関係は次の通りである。

Gyro Logic = Theory

GyroOS = Execution System

GyroAuth = Authentication Application

本研究は、Gyro Logic Core である Structure → Slice → Stability や GyroOS runtime contract を再定義しない。拡張対象は GyroAuth application layer の評価範囲である。

基礎 GyroAuth 論文は認証モデルそのものを定義する [14], [15]。Trajectory-Based Vulnerability Response 論文は、ログイン後の操作と Security response へ Trajectory 評価を適用する [16], [17]。本稿は、Authentication Criterion およびその更新過程の Integrity を対象とする。

2.2 Digital authentication と Zero Trust

NIST SP 800-63-4 および SP 800-63B-4 は、Digital identity と Authenticator management の要件を定める [1], [2]。本モデルは、Password、Passkey、MFA、Authenticator、Assurance level を置き換えない。Authenticator 結果や Challenge 結果は Evidence として利用できる。

NIST Zero Trust Architecture は、Network location に基づく暗黙の信頼を排し、Policy に基づく継続的な評価を重視する [3], [4]。GyroAuth はこの Architecture と両立するが、本稿が扱うのは、適応型認証 Component が将来の判断 Criterion を変更してよいかという、より限定された問題である。

2.3 Continuous authentication と Behavioral biometrics

Continuous authentication 研究は、Behavior、Biometric、Sensor、Contextual signal を用いて Session 中の利用者を継続的に評価する [5], [6]。GyroAuth も認証を一回の Login event へ還元しない。

ただし、本稿は次を分離する。

continuous subject evaluation

!=

permission to update the future criterion

Behavioral biometrics は Evidence となり得るが、Behavioral feature は Identity そのものではなく、Observed profile change は自動的に受理された Criterion update ではない。

2.4 Concept drift と Poisoning

Concept drift 研究は、時間とともに変化する Data distribution への適応を扱う [8]。これらの手法は Criterion Update Candidate の生成に利用できるが、Security-sensitive な用途では、観測された Drift をすべて正当な変化と仮定できない。

Poisoning 研究は、攻撃者が Training data や Adaptation data を操作し、将来の Model behavior を変更できることを示している [10], [13]。本稿では、この問題を Authentication 固有の状態遷移問題として扱い、Current subject decision、Future criterion adoption、Criterion state、Update response、Rollback linkage を分離する。

したがって、新規性主張は次の範囲に限定する。

本研究は、Adaptive authentication や Continuous authentication そのものの新規性を主張しない。提案の貢献は、Authentication Criterion の変更候補を Guard 付きで追跡可能な Trajectory として表し、現在の認証判断とは独立して将来の Criterion 変更を許可する Criterion Integrity model にある。

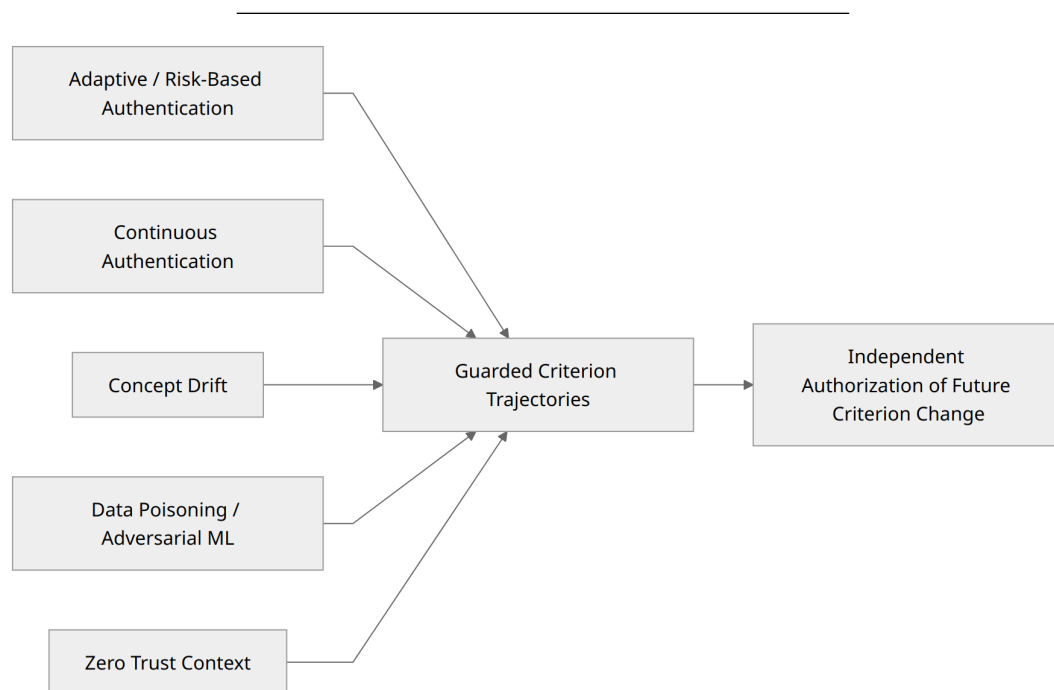


図 2. 本提案は、Adaptive Authentication、Continuous Authentication、Drift 対応、および Poisoning-aware adaptation の交点に位置づけられる。

3 用語と適用範囲

3.1 Access Subject

Access Subject は、現在評価されているアクセスおよび操作の発生源である。正当な利用者、Credential thief、Session hijacker、Relay-mediated operator、Bot、Remote controller、または混在した発生源で

あり得る。

Access Subject

!=

Verified Identity

3.2 Expected Identity

Expected Identity は、変化する Session や Context をまたいで、Access Subject を同一の認証 Identity として評価する際の持続的な参照関係である。Credential、端末、場所、行動は Evidence となり得るが、Identity そのものではない。

3.3 Observed Access Trajectory

Observed Access Trajectory は、現在の Orientation、Context、Slice のもとで Local Authentication Realization 間の許容可能な関係を辿ることによって得られる、可読な関係的構成である。

History

!=

Trajectory

3.4 Authentication Criterion

Authentication Criterion は、Evidence、Deviation、Stability、Trajectory、History、および Response 条件を解釈するために用いる Context-relative な評価基準である。

Authentication Criterion

!=

Fixed Identity Profile

3.5 Criterion Trajectory と Criterion Integrity

Criterion Trajectory は、Criterion 変化を追跡する関係的構成であり、更新理由、Evidence provenance、変化量、方向、速度、識別能力への影響、Response、Rollback linkage を含む。

Criterion Integrity は、Authentication Criterion とその更新過程が、Authentication Relation を評価する基準として、許容可能かつ追跡可能な状態を維持していることである。これは自己証明を意味せず、Protected Anchor、独立した Evidence、保持された History、Verified rollback point に依存する。

4 Threat Model

本モデルの保護対象は次である。

Authentication Relation
Authentication Criterion
Criterion Update Process
Criterion Trajectory
Trusted History
Rollback Points
Decision separation

Threat class には、Credential theft、Session hijacking、Relay attack、Gradual behavioral mimicry、Criterion poisoning、Evidence-source compromise、Coordinated multi-source compromise を含む。

Criterion poisoning とは、攻撃者が Evidence または Observed Access Trajectory へ影響し、不正な状態や操作を段階的に Effective Authentication Criterion へ取り込ませる攻撃である。

実行検証の中心は Gradual region-expansion poisoning である。その他、Criterion translation、Evidence-priority poisoning、Challenge weakening、Context-rule poisoning、History-window poisoning、Rollback-link poisoning、Slow contraction、Response-policy poisoning を Threat Model へ含める。

最小の Trust assumption は、少なくとも一つの Independent Evidence source、Protected policy anchor、Intact audit linkage、Verified rollback point が攻撃者の同時支配外に残ることである。すべての Evidence、Anchor、History、Rollback point が侵害された状況での識別や回復は保証しない。

5 Formal Security Model

評価は有限な Stage で進む。

$t = 0, 1, 2, \dots$

時点 t の Effective Criterion を A_t とする。Candidate は次で生成される。

$$\begin{aligned} A^*(t+1) \\ = \\ U(A_t, T_t^{\text{obs}}, C_{t+1}, E_t, H_t) \end{aligned}$$

Candidate generation は Adoption ではない。

$$\begin{aligned} A^*(t+1) \\ \neq \\ A_{t+1} \end{aligned}$$

Candidate は次の Guard で評価される。

$$\begin{aligned} G_t \\ = \\ \text{Guard}(A_t, A^*(t+1), T_t^{\text{obs}}, C_{t+1}, E_t, H_t, P_t) \end{aligned}$$

最小 Guard vector は次を評価する。

provenance
cross-evidence consistency
challenge confirmation
update magnitude
update rate
update direction
discrimination preservation
rollback linkage
Evidence-source integrity

Critical failure は平均値で相殺しない。

CriticalGuardFail

→

Criterion Update Response != ACCEPT

Response は次で選択する。

$D_{crit_t} = Pi_{crit}(G_t, Q_t, H_t)$

Effective Criterion の遷移は次である。

$A_{t+1} = A^*_{t+1}$ when ACCEPT
 $A_{t+1} = A_t$ when DEFER / FREEZE / REVIEW
 $A_{t+1} = A_{tau}$ when ROLLBACK, $tau < t$

Subject Evaluation は独立して次を選択する。

AUTH_STABLE
RECONVERGING
REAUTH_REQUIRED
AUTH_FAIL

Criterion Update Response は次を選択する。

ACCEPT
DEFER
FREEZE
REVIEW
ROLLBACK

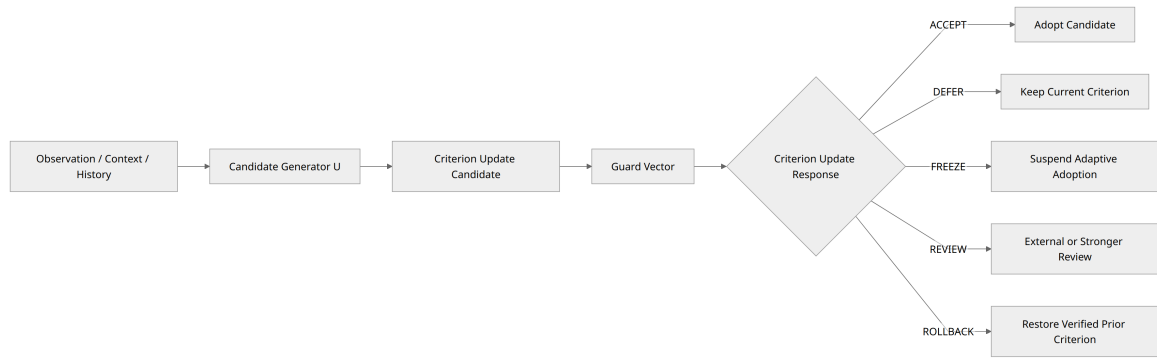


図 3. Candidate 生成と Candidate 採用を分離する。Candidate を有効化するのは ACCEPT のみである。

6 Criterion Update State Machine

Criterion State は次である。

STABLE

ADAPTING

UNCERTAIN

FROZEN

UNDER_REVIEW

COMPROMISED

ROLLED_BACK

Response semantics は次の通りである。

- ACCEPT: Candidate を次の Effective Criterion として採用する。
- DEFER: 追加 Evidence や Challenge 結果を待ち、現在の Criterion を維持する。
- FREEZE: Update path 自体が危険であるため Adaptive adoption を停止する。Subject Evaluation は独立して継続できる。
- REVIEW: 判断を External policy、Administrator、Stronger verification path、Independent validator へ移す。
- ROLLBACK: Audit trail を削除せず、Verified prior criterion へ戻す。

DEFER != FREEZE

REVIEW != DEFER

図の Source は figures/guarded_criterion_trajectories_mermaid.md に格納する。

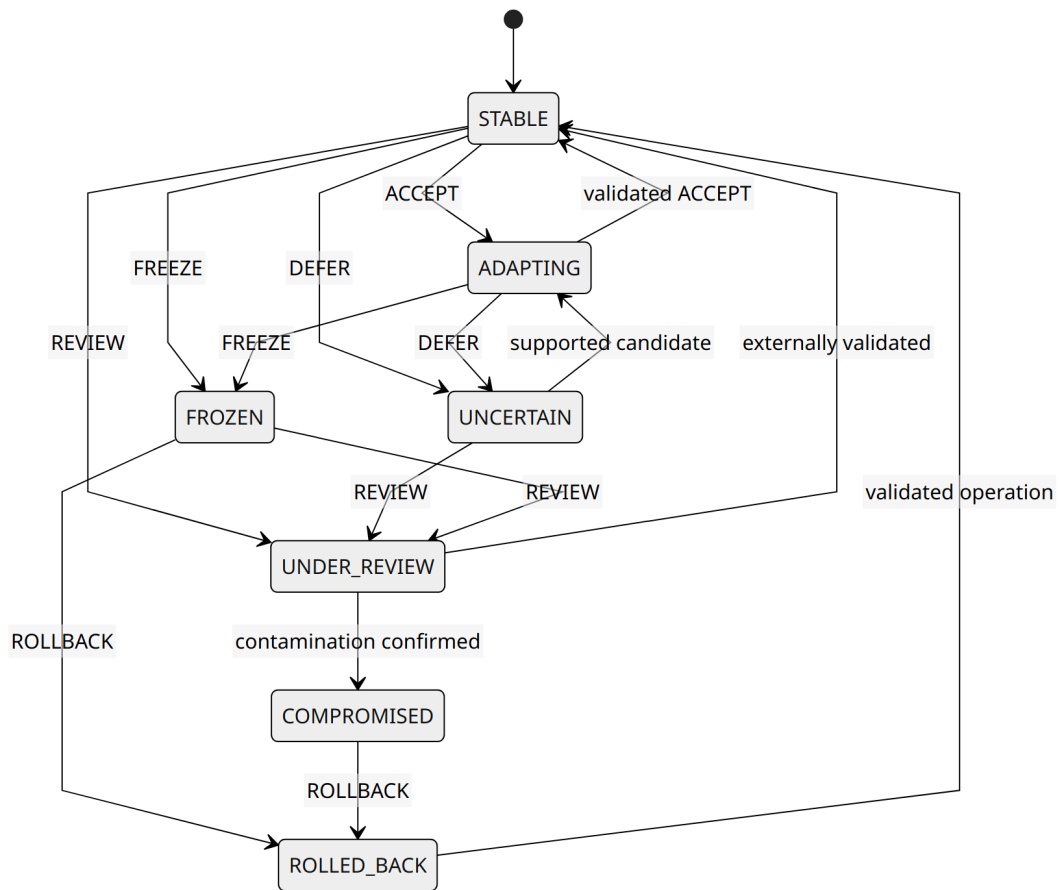


図 4. Criterion State と Criterion Update Response は異なる。FREEZE は Subject Evaluation を必ずしも終了させず、適応経路を停止する。

7 PoC 設計

PoC は同一の Candidate generator を用いる二つの Model を比較する。

7.1 Model U: Direct adoption

Observation

→ Candidate

→ Direct Adoption

7.2 Model G: Guarded adoption

Observation

→ Candidate

→ Guard

→ Criterion Update Response

→ Effective Criterion Transition

最小 Criterion 表現は次である。

```
A_t = (  
    mu_t,  
    width_t,  
    provenance_requirement_t,  
    challenge_requirement_t,  
    rollback_integrity_t  
)
```

実装は Python standard library のみを使用し、Synthetic かつ Deterministic な Input を用いる。

```
scripts/simulate_guarded_criterion_update.py  
examples/criterion_update/scenarios.json  
results/criterion_update_summary.json
```

Scenario は次である。

N1: Legitimate New Device Transition
P1: Gradual Region Expansion Poisoning
C1: Single Evidence Source Compromise

8 結果

8.1 N1: 正当な新規端末移行

Guarded model は次を出力した。

Auth Decision:
REAUTH_REQUIRED
→ AUTH_STABLE
→ AUTH_STABLE

Criterion Update Response:
DEFER
→ ACCEPT
→ ACCEPT

最終値は次である。

mu = 0.234
width = 0.120

Challenge confirmation 前の Candidate は保留され、Re-authentication 成功と Cross-evidence support の後に、Admissible region を拡張しない限定的な更新が採用された。

8.2 P1: 段階的な許容領域拡張 Poisoning

Direct-update baseline の最終値は次である。

```
final mu    = 0.3969728
final width = 0.277212
attack reference admissible = true
```

Guarded model の最終値は次である。

```
final mu    = 0.200
final width = 0.120
freeze stage = 2
attack reference admissible = false
```

Response path は次である。

```
DEFER
→ FREEZE
→ FREEZE
→ FREEZE
→ FREEZE
```

Stage 2 で次を出力した。

```
AUTH_STABLE + FREEZE
```

これは、Current subject acceptance と Permission to redefine future acceptance を独立して表現できることを示す。

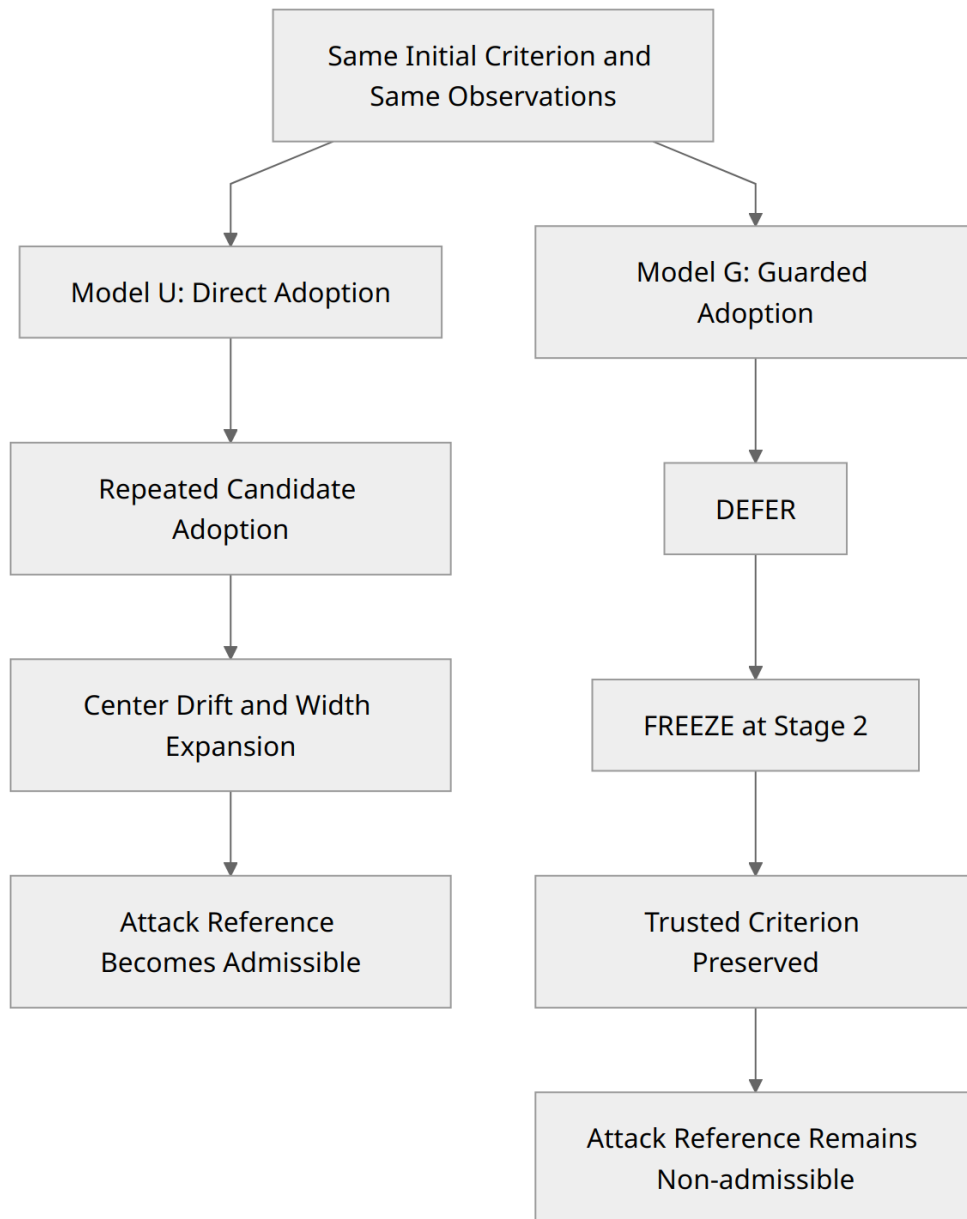


図 5. 実装した P1 Scenario では、直接採用は攻撃参照値が許容されるまで Criterion を拡張した一方、Guard 付き採用は許容前に適応を凍結した。

8.3 C1: 単一 Evidence source 侵害

Guarded model は次を出力した。

FREEZE

→ FREEZE

最終値は次を維持した。

$\mu = 0.200$

$\text{width} = 0.120$

Source integrity と Cross-evidence consistency が低いため、見かけ上強い Evidence が存在しても Candidate は採用されなかった。

実装した 7 件の Assertion はすべて成功した。

9 考察

中心的な結果は、Current Authentication Relation を継続する判断と、将来の Authentication Criterion を変更する判断を、実行可能な別判断として表現できることである。AUTH_STABLE + FREEZE がその最小例である。

本モデルは非適応ではない。N1 は、Challenge confirmation、Provenance check、Cross-evidence consistency を満たした正当な Context 変化を採用できることを示した。一方、観測の反復だけでは正当性を意味しない。

repeated observation
!=
new normal automatically

Criterion Integrity は自己証明ではない。External または Protected reference に依存する。また、Guard は単純な Weighted score ではなく、Critical failure を平均値で相殺しない。

Decision and Criterion Update Response are indepen

		Criterion update blocked	Criterion update allowed
Current relation	accepted	AUTH_STABLE + FREEZE	AUTH_STABLE + ACCEPT
	rejected	AUTH_FAIL + ROLLBACK	AUTH_FAIL + ACCEPT is invalid

図 6. AUTH_STABLE + FREEZE は、現在の Authentication Relation を一時的に継続しながら、Criterion 適応を禁止する状態を表す。

10 Security Claims と限界

実装した Deterministic assumption の範囲では、次を構造的に示した。

1. Candidate generation と Adoption を分離できる。
2. Auth Decision と Criterion Update Response を分離できる。
3. Support された Bounded adaptation を採用できる。
4. Direct adoption は Cumulative criterion expansion を生み得る。
5. Guarded adoption は実装した P1 で Attack-reference admission 前に Freeze できる。
6. C1 で Source-integrity failure が Adoption を阻止できる。
7. AUTH_STABLE + FREEZE を実行できる。

本研究は、Criterion poisoning の完全防止、Credential theft や Relay attack の普遍的検知、False Accept ゼロ、False Reject ゼロ、完全な Identity proof、Total compromise 下の Security、Production performance、Privacy guarantee、Formal correctness、Statistical generalization を示していない。

PoC は Synthetic かつ Deterministic であり、一次元 Criterion、縮約 Trajectory proxy、手動設定 Threshold を用いる。Real-world error rate や Operational cost は未測定である。

11 再現性と公開物

実行例は次である。

```
python scripts/simulate_guarded_criterion_update.py \  
  --scenarios examples/criterion_update/scenarios.json \  
  --output results/criterion_update_results.json
```

Repository には、Simulation source、Scenario input、Summary result、Formalization document、Figure source を含む。

12 結論

適応型認証は正当な変化に対応できる Criterion を必要とするが、Observation から Criterion への直接取り込みは Poisoning surface を生む。本稿は、Criterion change を Guard 付き Trajectory として扱う GyroAuth 拡張を提案した。Current Auth Decision と Future Criterion Update Response を独立して選択し、Deterministic PoC により、Bounded adaptation、実装した P1 条件下での Poisoning containment、Compromised-source update の阻止を示した。

中心命題は次である。

dynamic criterion

!=

unconstrained self-update

13 参考文献

- [1] D. Temoshok et al., Digital Identity Guidelines: Authentication and Authenticator Management, NIST SP 800-63B-4, 2025. DOI: 10.6028/NIST.SP.800-63b-4.
 - [2] D. Temoshok et al., Digital Identity Guidelines, NIST SP 800-63-4, 2025. DOI: 10.6028/NIST.SP.800-63-4.
 - [3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, Zero Trust Architecture, NIST SP 800-207, 2020. DOI: 10.6028/NIST.SP.800-207.
 - [4] R. Chandramouli and Z. Butcher, A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Cloud Environments, NIST SP 800-207A, 2023. DOI: 10.6028/NIST.SP.800-207A.
 - [5] M. Abuhamad, A. Abusnaina, D. Nyang, and D. Mohaisen, “Sensor-Based Continuous Authentication of Smartphones’ Users Using Behavioral Biometrics: A Contemporary Survey,” IEEE Internet of Things Journal, vol. 8, no. 1, pp. 65 – 84, 2021. DOI: 10.1109/JIOT.2020.3020076.
 - [6] A. Al Abdulwahid, N. Clarke, I. Stengel, S. Furnell, and C. Reich, “Security, Privacy, and Usability in Continuous Authentication: A Survey,” Sensors, vol. 21, no. 17, 5967, 2021. DOI: 10.3390/s21175967.
 - [8] J. Gama, I. Aliobait, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A Survey on Concept Drift Adaptation,” ACM Computing Surveys, vol. 46, no. 4, Article 44, 2014. DOI: 10.1145/2523813.
 - [10] B. Biggio, B. Nelson, and P. Laskov, “Poisoning Attacks against Support Vector Machines,” in Proceedings of the 29th International Conference on Machine Learning, 2012, pp. 1467 – 1474.
 - [13] A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies, and M. Hamin, Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations, NIST AI 100-2e2025, 2025. DOI: 10.6028/NIST.AI.100-2e2025.
 - [14] S. Kawakami, “GyroAuth: Authentication as Stability-Based Selection over State Convergence,” Jxiv, DOI: 10.51094/jxiv.4600.
 - [15] S. Kawakami, “GyroAuth :状態収束に対する安定性に基づく認証,” Jxiv, DOI: 10.51094/jxiv.5341.
 - [16] S. Kawakami, “Trajectory-Based Vulnerability Response,” Jxiv, DOI: 10.51094/jxiv.5416.
 - [17] S. Kawakami, “Trajectory に基づく脆弱性対応,” Jxiv, DOI: 10.51094/jxiv.5440.
-

14 AI 支援ツールの使用

本稿の構成整理、草稿作成補助、表現調整、整合性確認に AI 支援ツールを使用した。著者は本文、主張、参考文献、最終原稿を確認・編集し、その内容に全責任を負う。

15 申告事項

利益相反。本研究に関して、開示すべき利益相反はない。

研究資金。本研究は外部資金による支援を受けていない。

倫理および個人データ。本概念実証は合成された決定論的入力を使用し、人を対象とするデータまたは個人の認証記録を使用していない。

コードおよびデータの公開。本研究で使用したソースコード、決定論的シナリオ入力、結果概要は公開 GyroAuth リポジトリで提供する。本研究は合成入力を使用し、個人の認証テレメトリを含まない。

AI 支援ツールの使用。構成整理、草稿作成補助、表現調整、整合性確認のために AI 支援ツールを使用した。本文、主張、参考文献および最終原稿は著者が確認・編集し、全責任を負う。